

week-8 Develop a program using malloc(), calloc(), realloc(), and free().
Monitor memory allocation behavior and identify memory leaks using Valgrind.

Code:

```
File: ng.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;
    int *arr;

    // 1. malloc() - allocate memory
    printf("1. Using malloc()...\n");
    arr = (int *)malloc(5 * sizeof(int));

    if (arr == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (i = 0; i < 5; i++) {
        arr[i] = (i + 1) * 10;
    }

    printf("Array after malloc(): ");
    for (i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // 2. calloc() - allocate and initialize memory
    printf("2. Using calloc()...\n");
    int *zeroArr = (int *)calloc(5, sizeof(int));

    if (zeroArr == NULL) {
        printf("Memory allocation failed.\n");
        free(arr);
        return 1;
    }

    printf("Array after calloc(): ");
    for (i = 0; i < 5; i++) {
        printf("%d ", zeroArr[i]);
    }
    printf("\n");

    // 3. realloc() - resize allocated memory
    printf("3. Using realloc()...\n");
    int *temp = (int *)realloc(arr, 10 * sizeof(int));

    if (temp == NULL) {
        printf("Reallocation failed.\n");
        free(arr);
        free(zeroArr);
        return 1;
    }

    arr = temp;

    for (i = 5; i < 10; i++) {
        arr[i] = (i + 1) * 10;
    }

    printf("Array after realloc(): ");
    for (i = 0; i < 10; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```
Last login: Sat Sep 19 08:15:43 on console
sreenidhiredy@Sreen-MacBook-Pro ~ % nano ng.c
sreenidhiredy@Sreen-MacBook-Pro ~ % gcc ng.c -o ng
sreenidhiredy@Sreen-MacBook-Pro ~ % ./ng
1. Using malloc()...
Array after malloc(): 10 20 30 40 50
2. Using calloc()...
Array after calloc(): 0 0 0 0 0
3. Using realloc()...
Array after realloc(): 10 20 30 40 50 60 70 80 90 100
4. Using free()...
Memory successfully released.
sreenidhiredy@Sreen-MacBook-Pro ~ %
```

Develop a program that demonstrates Copy-on-Write behavior after a fork().
Analyze memory usage before and after modifying data in the child process.

Code:

```
1 238.c
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <unistd.h>
6 #include <sys/types.h>
7 #include <sys/wait.h>
8 #include <string.h>
9
10 #define SIZE (28 * 1024 * 1024) // 28 MB
11
12 // Global array so memory is inherited by child after fork()
13 char data[SIZE];
14
15 void show_memory(const char *name)
16 {
17     char path[100];
18     char line[256];
19
20     sprintf(path, "/proc/%d/status", getpid());
21
22     FILE *fp = fopen(path, "r");
23
24     if (fp == NULL) {
25         perror("fopen");
26         return;
27     }
28
29     printf("\n%s (PID = %d)\n", name, getpid());
30
31     while (fgets(line, sizeof(line), fp)) {
32         if (strncmp(line, "Vmalloc:", 7) == 0 ||
33             strncmp(line, "VmRSS:", 7) == 0) {
34             printf("%s", line);
35         }
36     }
37
38     fclose(fp);
39 }
40
41 int main()
42 {
43     pid_t pid;
44
45     printf("Parent process started.\n");
46
47     // Write to every page so the memory is actually allocated.
48     for (int i = 0; i < SIZE; i += 4096) {
49         data[i] = 'A';
50     }
51
52     printf("\n--- BEFORE fork() ---");
53     show_memory("Parent");
54
55     pid = fork();
56
57     if (pid < 0) {
58         perror("fork failed");
59         return 1;
60     }
61
62     if (pid == 0) {
63         // Child process
64         printf("\n\n--- CHILD PROCESS ---");
65
66         printf("\nBefore modifying memory:");
67         show_memory("Child");
68
69         sleep(2);
70
71         printf("\nChild is modifying memory...");
72     }
73
74     // Parent process
75     printf("\n\n--- PARENT PROCESS ---");
76     show_memory("Parent");
77
78     // Wait for child to finish
79     waitpid(pid, NULL, 0);
80
81     printf("\nChild process has terminated.\n");
82
83     return 0;
84 }
```

Output:

```
1 238.c
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <unistd.h>
6 #include <sys/types.h>
7 #include <sys/wait.h>
8 #include <string.h>
9
10 #define SIZE (28 * 1024 * 1024) // 28 MB
11
12 // Global array so memory is inherited by child after fork()
13 char data[SIZE];
14
15 void show_memory(const char *name)
16 {
17     char path[100];
18     char line[256];
19
20     sprintf(path, "/proc/%d/status", getpid());
21
22     FILE *fp = fopen(path, "r");
23
24     if (fp == NULL) {
25         perror("fopen");
26         return;
27     }
28
29     printf("\n%s (PID = %d)\n", name, getpid());
30
31     while (fgets(line, sizeof(line), fp)) {
32         if (strncmp(line, "Vmalloc:", 7) == 0 ||
33             strncmp(line, "VmRSS:", 7) == 0) {
34             printf("%s", line);
35         }
36     }
37
38     fclose(fp);
39 }
40
41 int main()
42 {
43     pid_t pid;
44
45     printf("Parent process started.\n");
46
47     // Write to every page so the memory is actually allocated.
48     for (int i = 0; i < SIZE; i += 4096) {
49         data[i] = 'A';
50     }
51
52     printf("\n--- BEFORE fork() ---");
53     show_memory("Parent");
54
55     pid = fork();
56
57     if (pid < 0) {
58         perror("fork failed");
59         return 1;
60     }
61
62     if (pid == 0) {
63         // Child process
64         printf("\n\n--- CHILD PROCESS ---");
65
66         printf("\nBefore modifying memory:");
67         show_memory("Child");
68
69         sleep(2);
70
71         printf("\nChild is modifying memory...");
72     }
73
74     // Parent process
75     printf("\n\n--- PARENT PROCESS ---");
76     show_memory("Parent");
77
78     // Wait for child to finish
79     waitpid(pid, NULL, 0);
80
81     printf("\nChild process has terminated.\n");
82
83     return 0;
84 }
```